

Introduction to Databases, 2<sup>e</sup>-B.Sc.

1<sup>ST</sup> semester 2023–2024, University of Antwerp

Prof. Dr. Toon Calders and M.Sc. Marco Favier



# Solutions Exam Introduction to Databases

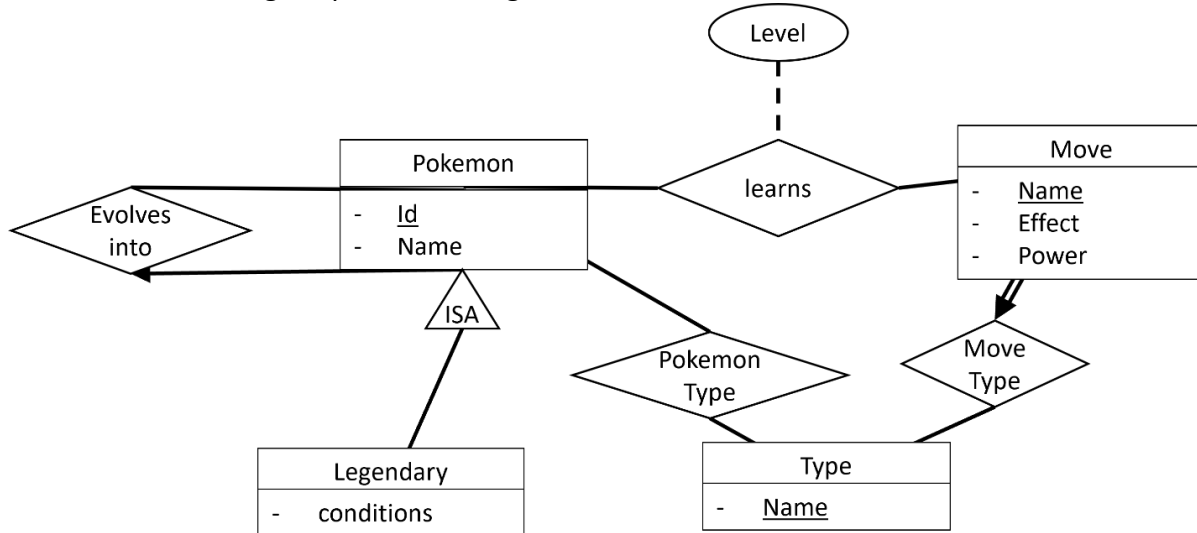
*01 February 2024*

*Solutions are given in red. Text in Italics indicates extra information; for instance on how grading was performed.*

*The exam took 3 hours and was closed-book.*

**Problem 1: Relational Model and Active Databases****(4 points)**

Consider the following simplified ER-Diagram for a database about Pokémon:



Pokémon is an RPG where collectible monsters fight each other. When a Pokémon reaches a specific level, it can learn a new move to fight. Once a Pokémon is strong enough it can transform into a new one, called an evolution. A Pokémon cannot be an evolution of two different Pokémon. Pokémon can have multiple types (like grass, water, etc.) while every move must have exactly one type.

A legendary Pokémon is a Pokémon that can be collected only if some conditions are met (like finishing the game).

a) **Relational Model:** Translate the diagram into a Relational Database

b) **Active Database:** Implement the following requirements into your database

- Level must be an integer always greater than 1 and lower than 100.
- Power has its own domain "pwr" and it is between 10 and 250.
- If a Pokémon is removed, then the followings happen:
  - If it has an evolution, we delete the evolution as well.
  - If it is legendary, we delete that entry as well.
  - Any move or type associated with that Pokémon is kept without the reference.
- If a Pokémon is modified, then we do the same at every reference.
- If a type is removed, we remove any move with that type.

*Part a was graded on 3 points, part b on 1 point.*

```

CREATE TABLE Pokemon (
    id INT primary key,
    name text not null,
    evolution_of INT references Pokemon on delete cascade on
update cascade
);

CREATE TABLE Legendary (
    id INT primary key references Pokemon on delete cascade on
update cascade,
    conditions text
);

CREATE TABLE Type (
    name text primary key
);

CREATE TABLE PokeType (
    id INT references Pokemon on update cascade on delete cascade,
    name text references Type,
    primary key (id, name)
);

CREATE DOMAIN pwr INT CHECK(value>=10 AND value <=250);

CREATE TABLE Move (
    name text Primary Key,
    effect text,
    power pwr,
    move_type text NOT NULL references Type on delete cascade
);

CREATE TABLE Learns (
    poke_id int references Pokemon on update cascade on delete
cascade,
    move_name text references Move,
    level INT check (level>0 and level<101),
    primary key (poke_id, move_name)
);

```

## Problem 2: SQL and Relational Algebra (6 points)

Consider the following schema for a plane travel database:

- Customer(fullname, birthplace) -- eg: ('Marco\_Favier', 'Milan')
- City(name, country) -- eg: ('Milan', 'Italy')
- Flight(f\_id, from\_city, to\_city, price) -- eg: (1001, 'Milan', 'Brussels', 150)
- Bought(fullname, f\_id) --eg: ('Marco\_Favier', 1001)

You can refer to the schema provided in the SQL database for the full set of constraints.

Write the following queries in Relational Algebra:

- a. Find the cities connected with a direct fly in one direction but not the other.

$\pi_{\text{from\_city, to\_city}} \text{Flight} - \pi_{\text{to\_city, from\_city}} \text{Flight}$

- b. For each direct connection between two cities find the price of the cheapest flight between them.

$\text{Connection} = \pi_{\text{from\_city, to\_city, price}} \text{Flight}$

$\text{Connection} - \pi_{\text{from\_city, to\_city, price}} \sigma_{\text{price} > \text{price2}} (\rho_{\text{price} \rightarrow \text{price2}} \text{Connection} \bowtie \text{Connection})$

- c. Find all customers that bought tickets to fly from their birthplace to every country in the dataset.

$\pi_{\text{fullname, country}} \sigma_{\text{name}=\text{to\_city}} \sigma_{\text{from\_city}=\text{birthplace}}$

$(\text{Customer} \bowtie \text{Bought} \bowtie \text{Flight} \times \text{City}) \div \pi_{\text{country}} \text{City}$

Write the following queries in SQL:

- a. Find all the couple of customers who have never bought a ticket for the same flight.

```
select c1.fullname, c2.fullname
from customer c1, customer c2
where not exists (
    select *
    from bought copy1, bought copy2
    where
        copy1.fullname = c1.fullname and
        copy2.fullname = c2.fullname and
        copy2.f_id = copy1.f_id
    ) and c1.fullname < c2.fullname;
```

- b. Find all the cycles of 3 cities that make a round trip. Print each cycle only once.

*Eg: If Milan->Brussels->Amsterdam->Milan is printed, then do not also print Brussels->Amsterdam->Milan->Brussels or Amsterdam->Milan->Brussels->Amsterdam.*

*Notice that Milan-> Amsterdam-> Brussels->Milan is still a valid solution instead.*

```
select distinct F1.from_city, F1.to_city, F2.to_city
from Flight F1, Flight F2, Flight F3
where
F1.to_city = F2.from_city and
F2.to_city = F3.from_city and
F3.to_city = F1.from_city and
F1.from_city < F1.to_city and
F1.from_city < F2.to_city;
```

- c. Give all customers that only buy the most expensive ticket available for each connection they take.

```
create view most_expensive as
select from_city, to_city, max(price)
from Flight
group by from_city, to_city;

select fullname
from Customer

except

select fullname
from Bought B, Flight F
where
    F.f_id=B.f_id AND
    not (F.from_city, F.to_city, F.price) in
    (
        select *
        from most_expensive
    );
```

You may find a database example on blackboard and at the following link:

Rel. Alg: <https://dbis-uibk.github.io/relax/calc/gist/ab63fd401fd9308a9d600d25d1371341>

*Each query was evaluated on 1 point.*

### Problem 3: Functional dependencies (4 points)

Consider the following relation schema and set of functional dependencies:

#### **Schedule(CourseCode, NameCourse, Lecturer, Program, Room, Weekday, Date, Timeslot)**

This table stores the course schedule of a faculty at the university. A program consists of courses, with course codes and lecturers. Lecturers teach courses in a room, on a date, in a timeslot. You may assume that different timeslots are not overlapping.

a) Express the following constraints as functional dependencies:

- Every course has a course name, and a unique course code. *Course names themselves, however, are not necessarily unique; for instance: the course “Introduction to programming” in the CS program and the course with the same name in the Math program are two different courses, with a different course code.*

$C \rightarrow N$

- A course name is unique within a program. *Notice that there are two courses “Introduction to programming”, but that’s fine because one is in the CS program, while the other is in the Math program.*

$PN \rightarrow C$

*Surprisingly many students got the answer wrong and wrote things like  $C \rightarrow P$  (a course can only be in one program), or even  $P \rightarrow CN$  (a program can only contain one course)*

- Only one course can take place at the same time (that is: on the same date, during the same timeslot) in the same room, and only one lecturer can be teaching in the same room at the same time. *This last part diverges from how it is at UAntwerp; in this question we explicitly disallow multiple teachers teaching in the same room at the same time. Nevertheless, the same course can be taught at the same time by different lecturers, as long as this is in different rooms.*

$DTR \rightarrow CL$

- A lecturer cannot teach two different courses at the same time (that is: on the same date during the same timeslot). Neither can she teach in two different rooms at the same time.

$DTL \rightarrow CR$

*A common mistake was:  $CDT \rightarrow LR$ , which means that at any point in time a course can be taught by one lecturer in one room. This is not what the assignment states; according to the assignment it is possible that, f.i., Daphne and Tom both teach IP at the same time in different rooms, so we have two tuples with the same course, date and time, but different rooms and lecturers.. But Daphne cannot teach both IP classes in different rooms at the same time. So, we cannot have two tuples with the same date, time, and lecturer, unless it is for the same course in the same room.*

- The date determines the weekday.

$D \rightarrow W$

*This question was graded on 1.5.*

- b) Give a lossless, dependency-preserving decomposition of the relation **Schedule** in 3NF, under the constraints in a). *If the FDs in a) are incorrect, only partial points will be awarded for b).*

We start by computing a minimal cover of the set of FDs:

$FD = \{ C \rightarrow N, PN \rightarrow C, DTR \rightarrow CL, DTL \rightarrow CR, D \rightarrow W \}$

From this set of FDs we can either remove C from the 3<sup>rd</sup> FD, or from the 4<sup>th</sup>, **but not both**.

It does not really matter which choice you make, the minimal set of relations will be: R<sub>1</sub>(PNC), R<sub>2</sub>(DTRLCL), R<sub>3</sub>(DW)

The only candidate keys in the relation Schedule are: DTPR and DTPL both of which are not included in any of the relations. So, one of these two relations needs to be added as well. In the end we have two solutions (if you follow the 3NF algorithm in the slides):

- R<sub>1</sub>(PNC), R<sub>2</sub>(DTRLCL), R<sub>3</sub>(DW), R<sub>4</sub>(DTPR)
- R<sub>1</sub>(PNC), R<sub>2</sub>(DTRLCL), R<sub>3</sub>(DW), R<sub>4</sub>(DTPL)

*Important parts in the decomposition: compute a minimal base, recombine again rules with the same left-hand-side, create a minimal number of relations (remove superfluous relations that are contained in other relations), add a key if necessary. To check if a key needs to be added it suffices to compute all candidate keys and check if at least one is contained. Alternatively the chase could be used and if the decomposition is not lossless, a key is added.*

*If the answer a) was wrong, the exercise part b) was graded as if the FDs found in a) were the right ones, but half a point was deducted. (of 1.5).*

- c) Show that your decomposition is **lossless**. *"I followed the algorithm in the slides" is not a correct answer; consider this question as an independent check that your answer is correct.*

We can use the chase to prove that our decomposition is lossless. We will do this only for option 1 of question b). We start with one tuple per relation:

| C | N | L | P | R | W | D | T |
|---|---|---|---|---|---|---|---|
| c | n |   | p |   |   |   |   |
| c |   | l |   | r |   | d | t |
|   |   |   |   |   | w | d |   |
|   |   |   | p | r |   | d | t |

Using  $DTR \rightarrow CL$ :

| C | N | L | P | R | W | D | T |
|---|---|---|---|---|---|---|---|
| c | n |   | p |   |   |   |   |
| c |   | l |   | r |   | d | t |
|   |   |   |   |   | w | d |   |
| c |   | l | p | r |   | d | t |

Using  $D \rightarrow W$ :

| C | N | L | P | R | W | D | T |
|---|---|---|---|---|---|---|---|
| c | n |   | p |   |   |   |   |
| c |   | l |   | r | w | d | t |
|   |   |   |   |   | w | d |   |
| c |   | l | p | r | w | d | t |

Using  $C \rightarrow N$ :

| C | N | L | P | R | W | D | T |
|---|---|---|---|---|---|---|---|
| c | n |   | p |   |   |   |   |
| c | n | l |   | r | w | d | t |
|   |   |   |   |   | w | d |   |
| c | n | l | p | r | w | d | t |

As one row is complete, we have obtained that the decomposition is lossless.

*This question was graded on 1 point.*

#### Problem 4: Normal forms (2 points)

True or false?

*Let  $R$  be a relation scheme and  $FD$  a set of functional dependencies.*

*Then  $R_1 \bowtie R_2$  is a lossless decomposition of  $R$  under  $FD$  if and only if:*

*at least one of the following is true:*

- $R_1 \cap R_2$  is a superkey of  $R_1$
- $R_1 \cap R_2$  is a superkey of  $R_2$

Provide a proof or a counter example.

*Answering True or False without a proper argumentation will not give you any points. **True.***

**If direction:** We assume WLOG that  $R_1 \cap R_2$  is a superkey of  $R_1$ . Hence  $R_1 \cap R_2 \rightarrow R_1$  holds. In the chase we can use this FD to complete the tuple of  $R_1$  and hence the decomposition is lossless.

**Only if direction:** Consider the chase algorithm for the decomposition  $R_1 \bowtie R_2$ . We show by induction that the only values that are changed by the chase correspond to attributes in  $(R_1 \cap R_2)^+$ .

Base case: as both tuples only agree on  $R_1 \cap R_2$ , the only FDs that can be used are of the form  $X \rightarrow Y$  with  $X \subseteq R_1 \cap R_2$ . The application of this FD changes only values for  $Y$ . As  $X \subseteq R_1 \cap R_2$ ,  $Y \subseteq (R_1 \cap R_2)^+$ .

Induction step: suppose only values in  $(R_1 \cap R_2)^+$  have changed. This implies that the only FDs that can be used are of the form  $X \rightarrow Y$  with  $X \subseteq (R_1 \cap R_2)^+$ . The application of this FD changes only values for  $Y$ . As  $X \subseteq (R_1 \cap R_2)^+$ ,  $Y \subseteq (R_1 \cap R_2)^+$ .

The decomposition is lossless, so one of the tuples is completed. We assume WLOG it is the first tuple.

This means that all values of  $R_2 \setminus (R_1 \cap R_2)$  have changed for this tuple and hence  $R_2 \setminus (R_1 \cap R_2) \subseteq (R_1 \cap R_2)^+$ .

Since also  $(R_1 \cap R_2) \subseteq (R_1 \cap R_2)^+$ ,  $R_2 \subseteq (R_1 \cap R_2)^+$ , which proves that  $R_1 \cap R_2$  is a superkey of  $R_2$ .

*This question was in the end graded as a bonus question; that is, the sum of the results for the other questions were rescaled to 20, and the result of question 4 was added to the final result.*



### Problem 5: Transaction management (3 points)

Consider the following schedules. In each schedule, T1 is started before T2, T2 before T3, ...

| <u>Schedule 1</u> |        |
|-------------------|--------|
| T1                | T2     |
| R(B)              |        |
|                   | R(B)   |
|                   | W(A)   |
| W(B)              |        |
|                   | R(C)   |
|                   | W(C)   |
|                   | commit |
| commit            |        |

| <u>Schedule 2</u> |        |        |
|-------------------|--------|--------|
| T1                | T2     | T3     |
| R(A)              |        |        |
|                   | R(B)   |        |
|                   |        | R(A)   |
| W(B)              |        |        |
|                   | W(A)   |        |
|                   |        | W(B)   |
|                   | commit |        |
| commit            |        |        |
|                   |        | commit |

| <u>Schedule 3</u> |        |        |
|-------------------|--------|--------|
| T1                | T2     | T3     |
| W(A)              |        |        |
|                   | R(C)   |        |
|                   | W(A)   |        |
|                   |        | W(A)   |
| W(B)              |        |        |
|                   | W(B)   |        |
|                   |        | W(B)   |
|                   |        | W(C)   |
| commit            |        |        |
|                   | commit |        |
|                   |        | commit |

a) Which of these schedules are conflict-serializable?

| <u>Schedule 1</u> |        |
|-------------------|--------|
| T1                | T2     |
| R(B)              |        |
|                   | R(B)   |
|                   | W(A)   |
| W(B)              |        |
|                   | R(C)   |
|                   | W(C)   |
|                   | commit |
| commit            |        |

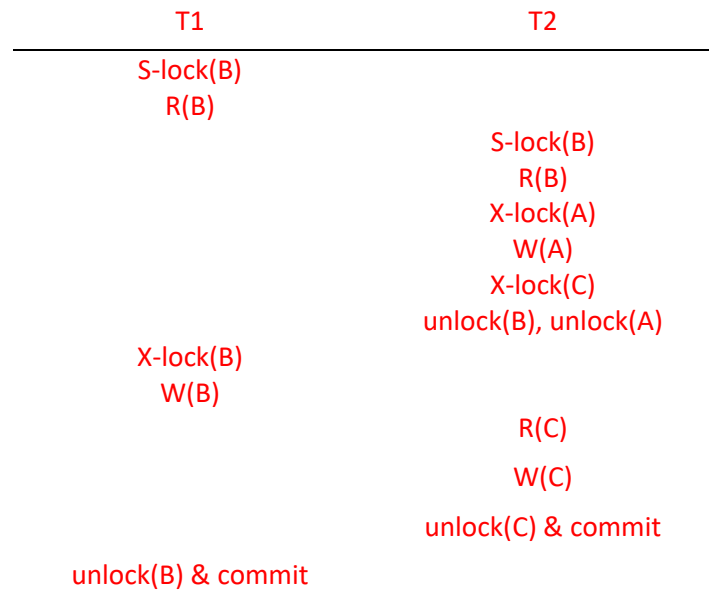
| <u>Schedule 2</u> |        |        |
|-------------------|--------|--------|
| T1                | T2     | T3     |
| R(A)              |        |        |
|                   | R(B)   |        |
|                   |        | R(A)   |
| W(B)              |        |        |
|                   | W(A)   |        |
|                   |        | W(B)   |
|                   | commit |        |
| commit            |        |        |
|                   |        | commit |

| <u>Schedule 3</u> |        |        |
|-------------------|--------|--------|
| T1                | T2     | T3     |
| W(A)              |        |        |
|                   | R(C)   |        |
|                   | W(A)   |        |
|                   |        | W(A)   |
| W(B)              |        |        |
|                   | W(B)   |        |
|                   |        | W(B)   |
|                   |        | W(C)   |
| commit            |        |        |
|                   | commit |        |
|                   |        | commit |

Above the conflicting instructions have been connected with a line. As can be seen, **only Schedule 2 is not conflict-serializable.**

b) Which of these schedules are possible under the 2PL protocol?

Schedule 2 can already be discarded, as 2PL generates only conflict-serializable schedules. For Schedule 1, it can be realized if the locks are obtained in the following order:



Schedule 3 is not possible under 2PL, because:

- T2 needs to have an exclusive lock on A to do W(A), and needs to release it before T3 can X-lock A to perform W(A).
- Hence, from step 4 on, T2 is in its shrinking phase.
- In step 6, T2 needs to have an X-lock on B to perform W(B).
- Combining the previous 2 bullets, T2 holds an X-lock on B from step 4 to 6
- This contradicts with T1 performing W(B) in step 5, which requires an X-lock on B.

- c) Which of these schedules are possible under the timestamp-based protocol without Thomas' write rule?

The timestamp based protocol without Thomas' write rule generates conflict-serializable schedules only, so schedule 2 can immediately be discarded. Furthermore, the timestamp-based protocol ensures that the result of the schedule is the same as if first T1 is entirely executed, then T2, then T3, ... ; that is, all conflicts are such that the instruction in the older transaction occurs first. This is clearly not the case in schedule 1, so not valid according to timestamp-based protocol. For the third schedule we trace the read and write timestamps step by step:

| R(A) | W(A) |  | R(B) | W(B) |  | R(C) | W(C) |
|------|------|--|------|------|--|------|------|
|      | 1    |  |      |      |  |      |      |
|      | 1    |  |      |      |  | 2    |      |
|      | 2    |  |      |      |  | 2    |      |
|      | 3    |  |      |      |  | 2    |      |
|      | 3    |  |      | 1    |  | 2    |      |
|      | 3    |  |      | 2    |  | 2    |      |
|      | 3    |  |      | 3    |  | 2    |      |
|      | 3    |  |      | 3    |  | 2    | 3    |

So, no conflicts in the time-stamp based protocol and hence schedule 3 is allowed.

*Many students did way too much work for this question. For instance, if a schedule is not conflict-serializable, the parts b and c could be skipped for that schedule.*

## Problem 6: Recovery (2 points)

In the ARIES recovery method as described in the course, it is very important that the database uses a transaction management protocol that disallows for an item written by an active transaction to be overwritten (red text in slide below). Why is this condition important? Given an example illustrating what can go wrong if this condition is not met.

### ARIES Recovery Method

- Recovery method for No Force – Steal
- Based on Write-ahead logging
- Also allows for roll-back of aborted transactions
- We will cover a highly simplified version
  - **No item written by an active transaction can be overwritten**; for instance **strict 2PL**
    - Always recoverability no cascading roll-backs
  - Only updates, no inserts or deletes
    - Database consists of items that can be overwritten



In the ARIES recovery method a log is maintained that stores for every write what was the previous value and what is the newly written value. Imagine now the following situation:

T1: W(A) A was 0, and becomes 1  
T2: W(A) A was 1, and becomes 2  
T1: abort

When we abort T1, we read the log to undo the writes of T1. This means that the value of 2, written by T2, is replaced by 0. This is incorrect, as the 1 written by T1 is already overwritten by T2. In principle we could alleviate the problem in this specific case, by checking that there is still a value of 1 in A. But then we fail in the following scenario:

T1: W(A) A was 0, and becomes 1  
T2: W(A) A was 1, and becomes 2  
T3: W(A) A was 2, becomes 1  
T1: abort

There is no simple solution for this problem except for carefully analyzing the log to figure out what the correct value should be. Note that such scenarios as sketched here, render the storage of an undo value useless. Hence, the ARIS algorithm covered in class is unable to deal with cases in which items written by active transactions can be overwritten.

*Important elements in the answer: logs contain undo values that become corrupted if a transactions overwrote a value written by a transaction that needs to be rolled back. Many students thought the problem was existence of cascading roll-backs or transactions not being recoverable. This was, however, not the essence of the problem. Non-recoverability needs to be dealt with by the concurrency protocol, and cascading roll-backs are not necessarily a problem on the one hand, and the problem may occur without cascading roll-backs on the other.*